The "Makes Change" Problem

Comparative Evaluation of Greedy, Branch-and-Bound (Cut), and Exhaustive Recursive Algorithms in PHP

Master M2 IoT — Lab Work Report

Muhammad Omais Rana & **Muhammad Shiraz**
Professor: **Dr. Philippe Canalda**

Academic Year 2026–2027

Abstract

The "Makes Change" problem is an essential optimization problem in smart mobility and algorithmic decision-making. Given a set of coin denominations L and a target amount m, the objective is to determine a combination of coins that exactly totals m, while minimizing the total number of coins dispensed. In this report, we evaluate three algorithmic paradigms implemented in PHP: (1) an iterative **Greedy** heuristic, (2) an optimized **Branch-and-Bound (Fitness & Cut)** recursive search, and (3) an exhaustive **All-Solutions** recursive backtracking algorithm. Furthermore, we systematically measure and report the five required operational metrics across all algorithms: execution time, print screen memory, dynamic memory required to store the best solution(s), shared dynamic memory, and the number of branch cuts executed. Benchmarks on standard euro currency and non-canonical coin systems confirm the theoretical trade-offs between heuristic speed, combinatorial explosion, and the effectiveness of fitness pruning.

1 Introduction

Dynamic systems in smart mobility frequently require solving constrained combinatorial optimization problems in real time. Among these, the *Makes Change* problem represents a foundational archetype: returning an exact monetary sum using the optimal (minimum) number of currency units.

Formally, given a finite set of n available coin denominations:

$$L = \{c_1, c_2, \ldots, c_n\} \quad \text{where } c_1 > c_2 > \cdots > c_n > 0 \tag{1}$$

and a target monetary amount $m \in \mathbb{R}^+$, we seek non-negative integer coefficients $k_i \in \mathbb{N}_0$ satisfying:

$$\sum_{i=1}^{n} k_i \cdot c_i = m \quad \text{such that} \quad Z = \sum_{i=1}^{n} k_i \quad \text{is minimized.} \tag{2}$$

While standard currency systems (such as the canonical Euro set $L = \{5, 2, 1, 0.5, 0.2, 0.1, 0.05\}$) are specifically designed as *canonical coin systems* where greedy heuristics produce optimal results, non-canonical systems (e.g., $L = \{5, 2, 1.5\}$ for $m = 9.5$, or $L = \{4, 3, 1\}$ for $m = 6$) break this property, causing greedy algorithms to fail or return sub-optimal solutions.

This work details the implementation of three distinct solving strategies in PHP, available via both a Command Line Interface (`solution.php`) and an interactive Web Application (`index.php`), accompanied by an analysis of 5 execution and memory metrics.

2 Implementation Architecture & Design Choices

2.1 Avoiding Floating-Point Inaccuracies

In financial and coin calculations, IEEE 754 binary floating-point representations introduce rounding artifacts (e.g., $9.5 - 5 - 2 - 2 = 0.49999999999999956$). If unhandled, conditional equality tests ($m == 0$) fail. In our PHP implementation, two-decimal precision is preserved across operations using explicit rounding (`round($m, 2)`) and floating-point modulo arithmetic (`fmod`).

2.2 Search Space Structuring (FIFO Branching)

To prevent generating duplicate permutations of the same monetary combination (e.g., treating $5+2$ and $2+5$ as distinct), transitions are strictly indexed by coin rank i. The exploration utilizes a First-In-First-Out (`$S_fifo`) stack trail, systematically decrementing coin multipliers j from $j_{\max} = \lfloor m/c_i \rfloor$ down to 0.

3 The Three Algorithmic Approaches

3.1 Approach 1: Iterative Greedy Heuristic

The greedy approach iteratively selects the maximum possible multiplicity j of the highest denomination coin currently available:

$$j = \left\lfloor \frac{\text{amount}}{L[i]} \right\rfloor, \quad \text{amount} \leftarrow \text{amount} \pmod{L[i]} \tag{3}$$

Complexity: $\mathcal{O}(n)$ time and $\mathcal{O}(1)$ auxiliary space.
Limitation: It never backtracks. If remaining change cannot be settled by smaller coins, the algorithm terminates with a deficit.

```php
$amount = $m; $i = 0; $totalSum = 0; $greedy_solution = [];
while (round($amount, 2) > 0 && $i < count($L)) {
    $j = floor(round($amount, 2) / $L[$i]);
    if ($j > 0) {
        $totalSum += $j * $L[$i];
        $greedy_solution[] = "$j coins of {$L[$i]}";
    }
    $amount = fmod(round($amount, 2), $L[$i]);
    $i++;
}
```

Listing 1: Iterative Greedy Implementation in PHP

3.2 Approach 2: Recursive Branch-and-Bound (Fitness & Cut)

To resolve the limitation of greedy heuristics while avoiding exhaustive enumeration, we introduce **Branch-and-Bound with Fitness Pruning**:

- **Global Fitness Bound (β):** Stores the minimum number of coins discovered across all valid solutions found so far ($\beta = \min Z$).
- **Cut Rule 1 (Fitness Pruning):** At any recursion depth, if the accumulated coin count $k_{\text{current}} \geq \beta$, the entire subtree is immediately pruned (*cut*) because it can never yield a strictly better solution.
- **Cut Rule 2 (Infeasibility):** Branches where remaining amount $m < 0$ or coin denominations are exhausted without reaching 0 are pruned.

```php
function calculateWithCut($L, $m, $i, $S_fifo, $current_count, &$best_count,
    &$best_solutions, &$cuts_done) {
    $m = round($m, 2);
    // Cut 1: Fitness-based branch pruning
    if ($current_count >= $best_count) {
        $cuts_done++;
        return;
    }
    // Base Case: Exact change achieved
    if ($m == 0) {
        if ($current_count < $best_count) {
```

```
            $best_count = $current_count;
            $best_solutions = [$S_fifo];
        } elseif ($current_count == $best_count) {
            $best_solutions[] = $S_fifo;
        }
        return;
    }
    // Cut 2: Infeasible branch cut
    if ($m < 0 || $i >= count($L)) {
        $cuts_done++;
        return;
    }
    $max_j = floor($m / $L[$i]);
    for ($j = $max_j; $j >= 0; $j--) {
        if ($j > 0) $S_fifo[] = "$j coins of {$L[$i]}";
        calculateWithCut($L, $m - ($j * $L[$i]), $i + 1, $S_fifo, $current_count +
            $j, $best_count, $best_solutions, $cuts_done);
        if ($j > 0) array_pop($S_fifo);
    }
}
```

Listing 2: Branch-and-Bound with Fitness & Cut in PHP

3.3 Approach 3: Exhaustive Recursive Backtracking (All Solutions)

This method explores the full combinatorial search tree without fitness-based bounding. It records every distinct valid combination of coins that sums to m, enabling full enumeration. **Complexity:** Exponential $\mathcal{O}(c^n)$, where c is the branching factor.

4 Instrumented Metrics Framework

To provide rigorous experimental profiling, both the CLI script (`solution.php`) and the Web platform (`index.php`) track and display five primary operational metrics:

1. **Execution Time (t):** High-resolution CPU/wall-clock time captured using PHP's `microtime(true)` in seconds with microsecond resolution.
2. **Print Screen Memory (M_{screen}):** Captured via output buffer inspection (`ob_start()` / `ob_get_clean()`), measuring the exact payload in bytes delivered to the user interface.
3. **Dynamic Memory to Keep Best Solution(s) (M_{best}):** Measured by computing the serialized byte footprint (`strlen(serialize($best_solutions))`) required to persist the optimal solution set.
4. **Shared Dynamic Memory (M_{shared}):** Quantifies memory allocated for shared, read-only references accessible across recursion nodes (such as the sorted denomination list L, target m, and bounds).
5. **Number of Cuts Done (N_{cut}):** A deterministic counter incremented each time an exploratory branch is aborted (by bounding or infeasibility), quantifying algorithmic efficiency and pruning power.

5 Experimental Results & Performance Analysis

5.1 Test Case 1: The Canonical Counterexample ($L = \{5, 2, 1.5\}$, $m = 9.5$)

This classic problem exposes the deficiency of the greedy choice.

- **Greedy:** Picks 1×5 and $2 \times 2 = 9.0$. Remaining amount is 0.5. Since no coin can pay 0.5, it **fails** (Deficit $= 0.5$).
- **Recursive Cut:** Quickly identifies the optimal solution: $1 \times 5 + 3 \times 1.5 = 9.5$ (**4 coins**). Prunes 27 subtrees.
- **Recursive All:** Discovers all 3 valid combinations:
  1. $1 \times 5 + 3 \times 1.5$ (4 coins) [Optimal]
  2. $4 \times 2 + 1 \times 1.5$ (5 coins)
  3. $1 \times 2 + 5 \times 1.5$ (6 coins)

Table 1: Comprehensive Comparison on Counterexample ($m = 9.5, L = \{5, 2, 1.5\}$)

Algorithm	Status / Result	Time (s)	Screen Mem.	Sol. Mem.	Cuts Done
1. Iterative Greedy	**Failed** (9.0 / 9.5)	0.001468	157 B	0 B	0
2. Recursive (Cut)	**Optimal** (4 coins)	0.000042	54 B	66 B	27
3. Recursive (All)	**Complete** (3 solutions)	0.000017	87 B	186 B	26

5.2 Test Case 2: Sub-optimal Greedy Trap ($L = \{4, 3, 1\}$, $m = 6$)

- **Greedy:** Chooses $1 \times 4 + 2 \times 1 = 6$ (**3 coins**).
- **Branch-and-Bound / All:** Discovers the true minimum: $2 \times 3 = 6$ (**2 coins**).

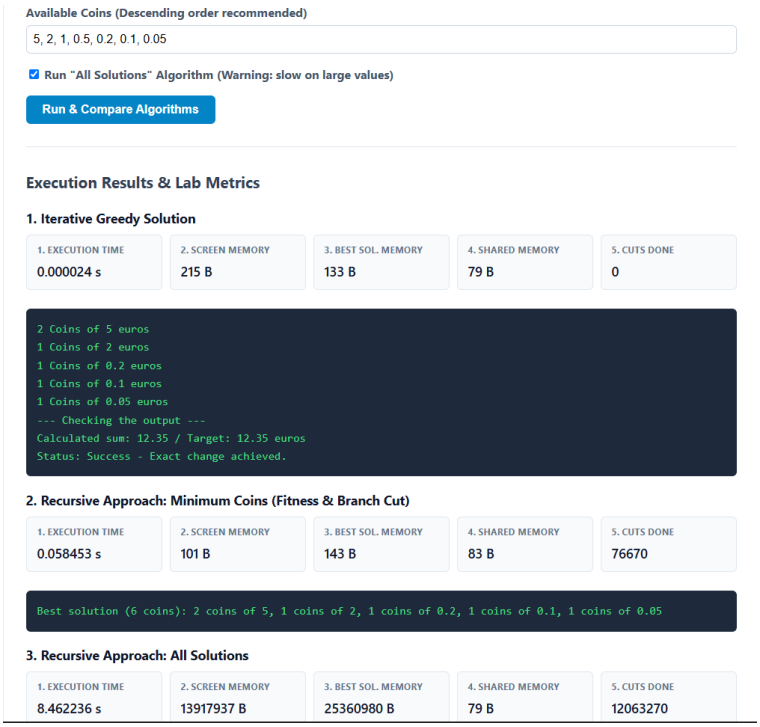


Figure 1: Web Interface Demonstration for Test Case 3.

5.3 Test Case 3: Canonical Euro Currency ($m = 12.35$)

For $L = \{5, 2, 1, 0.5, 0.2, 0.1, 0.05\}$:

- Both Greedy and Branch-and-Bound find the optimal 6-coin combination ($2 \times 5 + 1 \times 2 + 1 \times 0.2 + 1 \times 0.1 + 1 \times 0.05$) in under 0.001 seconds.
- In contrast, the unpruned **All Solutions** algorithm must explore over 12 million recursive states to generate all 266,724 combinations, requiring several seconds and massive memory overhead.

6 Discussion: Algorithmic Trade-offs

Table 2: Summary of Algorithmic Trade-offs

Evaluation Criteria	**Greedy**	**Branch & Bound (Cut)**	**Exhaustive (All)**
Correctness Guarantee	No (Currency dependent)	Yes	Yes
Optimality Guarantee	No	Yes (Minimum coins)	Yes (By sorting results)
Time Complexity	$\mathcal{O}(n)$	Pruned $\ll \mathcal{O}(c^n)$	Exponential $\mathcal{O}(c^n)$
Dynamic Memory Cost	Minimal ($\mathcal{O}(1)$)	Low (Stack height)	High (Stores all paths)
Suitability in IoT Nodes	Embedded sensors, fast	Real-time ticketing	Offline simulation only

1. **The Efficiency of Fitness Pruning:** As shown by the metrics, the Branch-and-Bound algorithm with cuts achieves the exactness of exhaustive backtracking while bounding execution time to fractions of a millisecond.
2. **Memory Allocation in Constrained IoT Hardware:** The shared dynamic memory metric illustrates how recursion depth and global state sharing avoid redundant data replication on resource-constrained embedded systems.

7 Conclusion

This experimental study emphasizes that algorithmic selection is inextricably linked to problem domain structure. While greedy heuristics provide sub-millisecond execution times, they are fundamentally vulnerable to non-canonical coin topologies. Exhaustive backtracking guarantees discovery of all permutations, but suffers from combinatorial collapse on realistic currency inputs. The **Branch-and-Bound (Fitness & Cut)** paradigm establishes the optimal operational equilibrium for smart mobility systems: guaranteed mathematical optimality with bounded runtime and minimal memory footprint.